
REDES NEURONALES 2023

Trabajo Final Integrador

Emmanuel Beltramo^{1,2}

¹*Instituto de Estudios Avanzados en Ingeniería y Tecnología (IDIT) - CONICET, UNC*

²*Departamento de Estructuras, FCEFyN, Universidad Nacional de Córdoba*

RESUMEN

En este trabajo integrador se construyen dos redes neuronales convolucionales que operan sobre el conjunto de datos Fashion-MNIST. Una de ellas es un autoencoder, mientras que la otra es un clasificador que reutiliza el encoder del autoencoder convolucional. En el informe se presentan y comentan los resultados obtenidos para ambas arquitecturas empleando diferentes hiperparámetros. En última instancia, se sintetizan los aspectos más relevantes.

1. INTRODUCCIÓN

En este trabajo se estudiaron dos redes neuronales diferentes de tipo convolucional: un autoencoder y un clasificador. Las dos arquitecturas fueron implementadas utilizando Pytorch. Con el propósito de encontrar la combinación de hiperparámetros que produzca los mejores resultados se consideraron, para ambos casos, las siguientes variables de ajuste:

- Un planificador de la tasa de aprendizaje de la red (CyclicLR en PyTorch) para acelerar la reducción del error con el número de épocas. Éste fue configurado tal que la tasa de aprendizaje varíe de manera triangular con valores ubicados entre 10^{-6} y 10^{-3} . El período de la onda triangular es de 10 minilotes.
- Diversos números de neuronas n para la capa lineal intermedia de ambas arquitecturas: se consideraron 64, 128, 256 y 512 neuronas.
- Minilotes de entrenamiento (ME) de diferentes tamaños, se optó por 100, 500, 1000 y 2000 muestras. A su vez, se probaron dos valores diferentes de dropout, $p = 0.2$ y $p = 0.4$.
- Distintas funciones de activación en la capa de salida. Se evaluaron las funciones Sigmoide, ReLU y LeakyReLU con pendiente negativa igual a 0.1.
- Dos optimizadores, el método del descenso por gradiente estocástico (SDG) y el método de Adam.

Se analizó el desempeño del autoencoder observando la evolución del error medio cuadrático promedio (EMC) con el número de épocas, así como su capacidad para reconstruir las imágenes usadas como entrada. Por su parte, el rendimiento del clasificador se estudió a partir de la evolución de la entropía cruzada (EC) y de la exactitud con el número de épocas. Además, para esta arquitectura se construyó la matriz de confusión.

2. RESULTADOS

2.1. Autoencoder

En primer lugar, se entrenó el autoencoder empleando los hiperparámetros dados por defecto. Al testear la red se observó que ciertas regiones de la imagen original no se reconstruían correctamente. En particular, como se muestra en la Figura 1a, tales regiones corresponden a los píxeles ubicados en las tres últimas filas y tres últimas columnas del lado derecho (ver recuadro rojo). Con el objetivo de resolver este inconveniente y

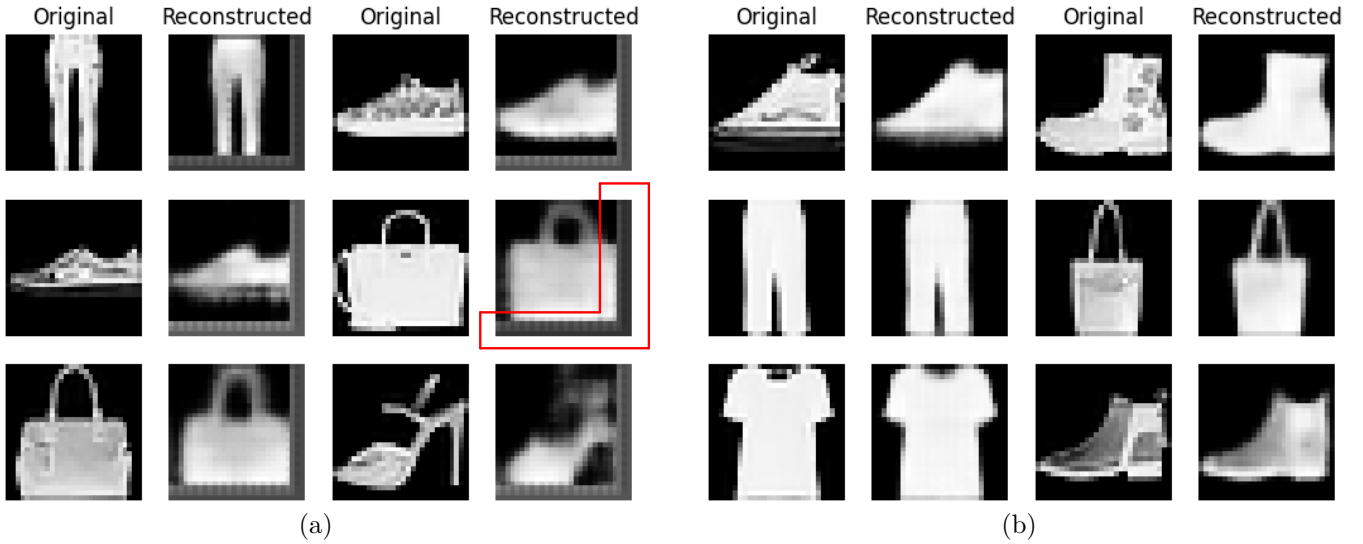


Figura 1: (a) Autoencoder original: seis imágenes junto a su reconstrucción. (b) Autoencoder modificado: seis imágenes junto a su reconstrucción (caso D, Figura 2).

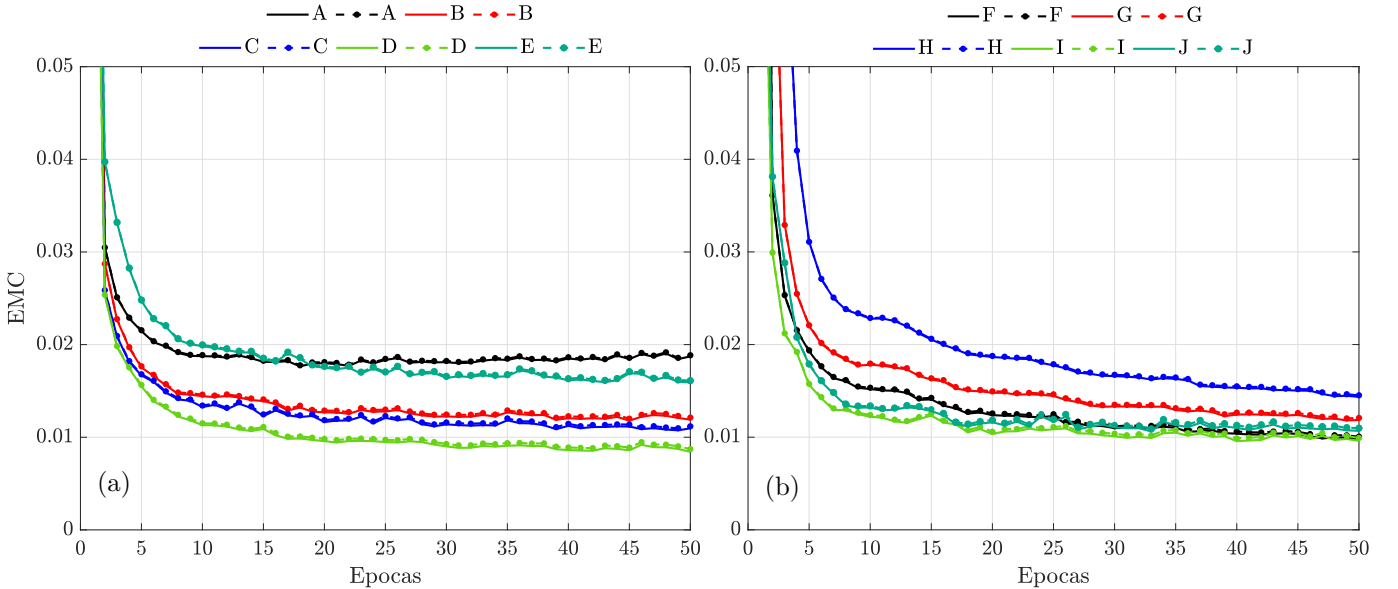


Figura 2: (a) A: $n = 64$, $p = 0.2$, Adam, ME de 100, Sigmoide. B: $n = 128$, $p = 0.2$, Adam, ME de 100, Sigmoide. C: $n = 256$, $p = 0.2$, Adam, ME de 100, Sigmoide. D: $n = 512$, $p = 0.2$, Adam, ME de 100, Sigmoide. E: $n = 64$, $p = 0.4$, Adam, ME de 100, Sigmoide. (b) F: $n = 512$, $p = 0.2$, Adam, ME de 500, Sigmoide. G: $n = 512$, $p = 0.2$, Adam, ME de 1000, Sigmoide. H: F: $n = 512$, $p = 0.2$, Adam, ME de 2000, Sigmoide. I: $n = 512$, $p = 0.2$, Adam, ME de 100, ReLU. J: $n = 512$, $p = 0.2$, Adam, ME de 100, LeakyReLU. Línea continua: conjunto de entrenamiento, línea a trazos: conjunto de testeo.

después de efectuar varias pruebas, se realizó una ligera modificación en la arquitectura del autoencoder. Se incorporó un **padding** de 1 tanto en las dos capas convolucionales del encoder como en las dos capas convolucionales transpuestas del decoder. Además, para la primera de estas últimas se modificó el **kernel_size** a un valor de 4. Finalmente, se evaluó el autoencoder con la arquitectura modificada.

En la Figura 2, se grafica el EMC en función del número de épocas para los conjuntos de entrenamiento y testeo, considerando diferentes combinaciones de hiperparámetros. Según se observa, los errores más bajos se obtienen con los casos D e I. Ambos tienen 512 neuronas en la capa lineal intermedia, 100 muestras en el minilote de entrenamiento y un dropout de 0.2. La diferencia entre ellos radica en la función de activación usada en la capa de salida, Sigmoide para el caso D y ReLU para el I. Aunque no pareciera existir sobreajuste, el número óptimo de épocas de entrenamiento se encuentra entorno a las 30, ya que a partir de este número

el EMC tiende hacia un valor prácticamente constante. En la Figura 1b se presentan seis imágenes originales junto a su reconstrucción utilizando el autoencoder del caso D. Por último, es importante mencionar que no se graficaron los EMCs obtenidos con el optimizador SDG, ya que los valores eran sustancialmente mayores en comparación con los casos estudiados para el mismo número de épocas de entrenamiento.

2.2. Clasificador

Para el clasificador, se utilizaron los pesos sinápticos del encoder del autoencoder como preentrenamiento. Posteriormente, se entrenó el clasificador con diferentes combinaciones de hiperparámetros.

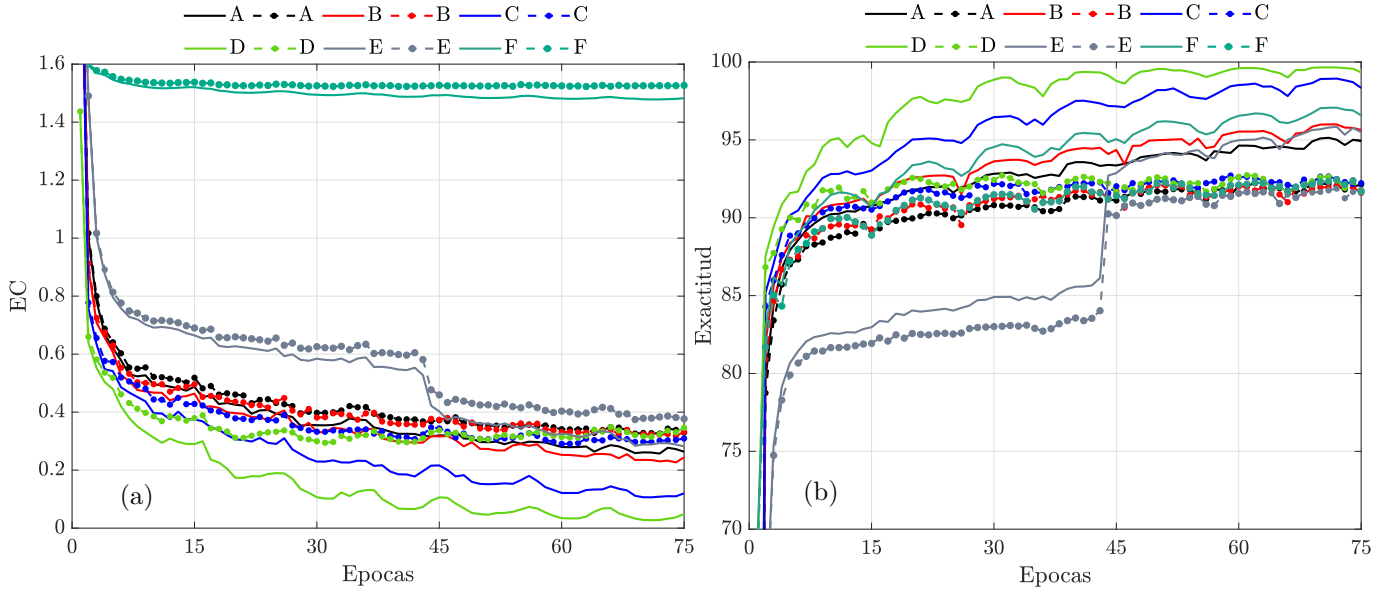


Figura 3: (a) Entropía cruzada en función del número de épocas y (b) exactitud en función del número de épocas para los siguientes casos. A: $n = 64$, $p = 0.2$, Adam, ME de 100, ReLU. B: $n = 128$, $p = 0.2$, Adam, ME de 100, ReLU. C: $n = 256$, $p = 0.2$, Adam, ME de 100, ReLU. D: $n = 512$, $p = 0.2$, Adam, ME de 100, ReLU. E: $n = 512$, $p = 0.2$, Adam, ME de 2000, ReLU. F: $n = 512$, $p = 0.2$, Adam, ME de 100, Sigmoide. Línea continua: conjunto de entrenamiento, línea a trazos: conjunto de testeo.

En la Figura 3a, se grafica la EC en función del número de épocas tanto para el conjunto de entrenamiento como el de testeo. Para el primero de ellos se observa que, en general, a medida que se incrementa el número de neuronas en la capa lineal, la EC converge hacia un valor más pequeño. Sin embargo, esto también acelera la aparición de sobreajuste cuando aumentan las épocas de entrenamiento. Para los casos evaluados, el número óptimo de épocas se ubica alrededor de 45, por encima de este valor es probable que ocurra sobreajuste, véase caso D. Es interesante notar que las curvas de EC asociadas al conjunto de testeo se encuentran próximas entre sí, independientemente de la cantidad de neuronas de la capa lineal. No obstante, existe una excepción en el caso F, donde la EC calculada para el conjunto de entrenamiento y de testeo toman valores sustancialmente mayores.

Por otra parte, en la Figura 3b se presenta la evolución de la exactitud para los dos conjuntos conforme aumentan las épocas de entrenamiento. De ello se deduce que, cuanto mayor es el número de neuronas en la capa lineal la exactitud evaluada sobre el conjunto de entrenamiento aumenta, pudiendo alcanzar valores cercanos al 99.5 %, tal como ocurre para el caso D. En cambio, si se tiene en cuenta lo que sucede para el conjunto de testeo se advierte que la exactitud es prácticamente igual para todos los casos analizados, la cual supera ligeramente el 90 %. Esto podría atribuirse al comportamiento que tiene la EC sobre el conjunto de testeo.

La Figura 4a presenta la EC y la exactitud en función del número de épocas para dos casos adicionales. Al comparar los valores numéricos de estas curvas con los análogos de la Figura 3, se puede notar que la EC posee valores más elevados y la exactitud magnitudes más pequeñas. Las curvas de EC del caso H muestran cierta convergencia, en tanto que las correspondientes al caso G requieren un número de épocas adicionales para alcanzar convergencia. Esto puede atribuirse al optimizador empleado, ya que en el caso G se adoptó

al método del SGD y en el caso H al método de Adam, ofreciendo este último mejores prestaciones.

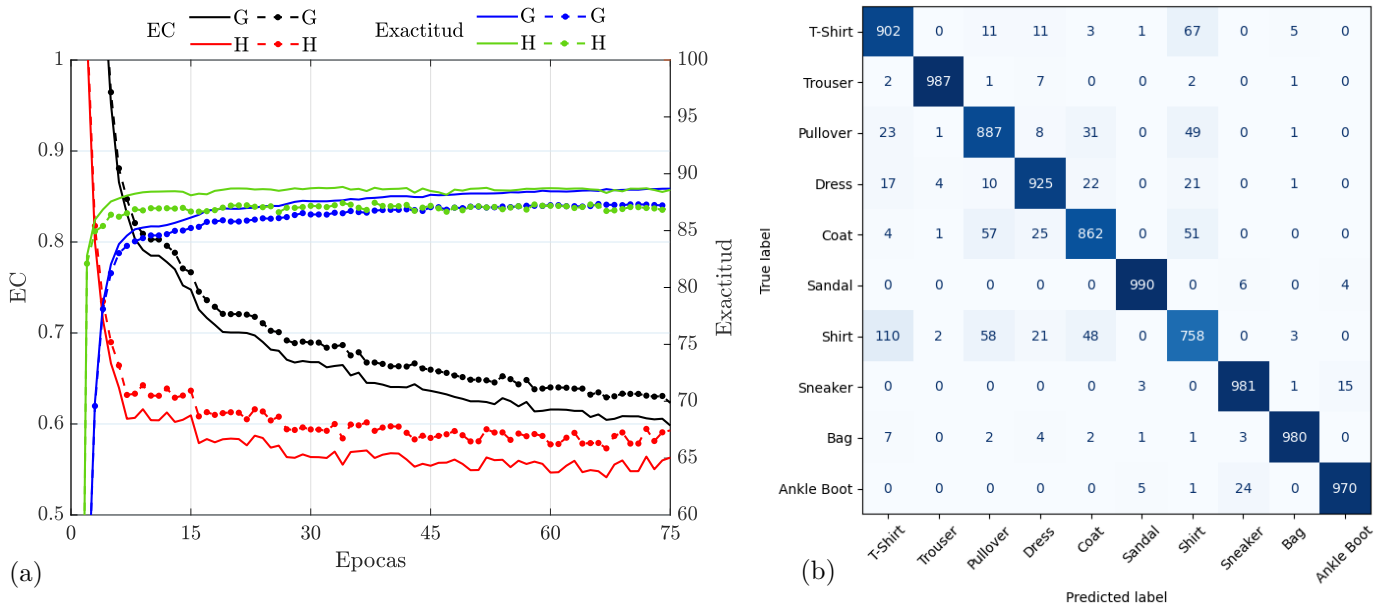


Figura 4: (a) Entropía cruzada y exactitud en función del número de épocas para los siguientes casos. G: $n = 512$, $p = 0.2$, SGD, ME de 100, ReLU. H: $n = 512$, $p = 0.2$, Adam, ME de 100, ReLU, pesos sinápticos del encoder con valores fijos. (b) Matriz de confusión calculada sobre el conjunto de testeo utilizando el clasificador del caso D de la Figura 3. Exactitud obtenida: 92 %. Línea continua: conjunto de entrenamiento, línea a trazos: conjunto de testeo.

De los casos analizados, se puede elegir como mejor alternativa al caso D entrenado durante 45 épocas para evitar el sobreajuste. Los hiperparámetros asociados son 512 neuronas en la capa lineal intermedia, función de activación ReLU en la capa de salida, un dropout $p = 0.2$, minilotes de 100 muestras para el entrenamiento y utilización del método de Adam como optimizador. En la Figura 4b se presenta la matriz de confusión para este caso, la cual se obtuvo a partir del conjunto de testeo compuesto por 10000 muestras. La exactitud alcanzada es de 92 %. Asimismo, se observa que los mayores errores de predicción aparecen cuando el clasificador debe identificar una camisa (*shirt*) y la confunde con una remeras mangas cortas (*t-shirt*). Lo mismo ocurre en sentido contrario pero con mayor probabilidad de acierto.

3. CONCLUSIONES

En este trabajo se implementaron, entrenaron y testearon dos redes neuronales de tipo convolucional, un autoencoder y un clasificador, usando el conjunto de datos Fashion-MNIST. Sobre ambas arquitecturas se probaron diversas combinaciones de hiperparámetros con el propósito de seleccionar la mejor de ellas. En lo que respecta al autoencoder, se realizó una pequeña modificación a la arquitectura inicial con el objetivo de mejorar su performance. Los errores mas bajos se obtuvieron empleando 512 neuronas en la capa lineal intermedia, un dropout de 0.2, el método de Adam como optimizador y la función Sigmoide para la capa de salida. Por otro lado, parte de la arquitectura del clasificador utilizó el encoder del autoencoder modificado y se aplicó el concepto de *transfer learning* para este bloque, es decir, se utilizaron sus pesos sinápticos como inicializadores. Los mejores resultados se obtuvieron con 512 neuronas en la capa lineal intermedia, con un dropout de 0.2, empleando el método de Adam para minimizar la función de pérdida y usando a la función ReLU para la capa de salida. La exactitud (*accuracy*) calculada sobre el conjunto de testeo se ubica en torno al 92 %, lo cual es un resultado aceptable.

BIBLIOGRAFÍA

- [1] Francisco A. Tamarit, *Material de Clases Teóricas – Redes Neuronales*, Facultad de Matemática, Física y Astronomía, Universidad Nacional de Córdoba, 2023.
- [2] PyTorch. <https://pytorch.org/>